

[L] 阮崇轩 · 前端开发工作汇报 [R] 转转 [C]1

前端开发技术报告

校园二手物品交易平台——转转

阮崇轩 · 前端开发

华中农业大学 · 2026 年 7 月

目录

1 概述	3
2 前端项目如何启动和运行	3
2.1 项目是怎么组织起来的	3
2.2 项目的文件结构	3
3 Axios 封装：所有网络请求的基础	4
3.1 什么是 Axios	4
3.2 为什么要封装	4
3.3 核心代码和解释	4
3.4 Token 自动刷新：最精巧的部分	5
4 登录与认证流程	5
4.1 完整登录流程	5
4.2 为什么需要两个 Token	6
4.3 登出时的安全处理	6
5 注册与验证码机制	7
5.1 注册流程	7
5.2 验证码的安全设计	7
5.3 验证码防消费的实现细节	7
6 消息系统	8
6.1 数据流	8
6.2 为什么用轮询而不是 WebSocket	8
6.3 轮询的具体实现	8
6.4 商品页联系卖家的实现	9
6.5 通知系统的实现	9
7 订单详情页面	10
7.1 角色判断：你怎么知道我是买家还是卖家	10
7.2 根据状态和角色决定显示什么	10
7.3 支付页面：模拟支付的四步流程	11
7.4 发货功能	11
8 后端 Bug 修复	12
8.1 验证码类型隔离（安全漏洞）	12
8.2 密码重置暴力破解防护	12

目录	2
8.3 浏览量统计错误	12
8.4 订单 VO 缺少字段	12
8.5 订单状态变更自动通知	12
9 页面动效的实现	13
9.1 骨架屏：加载中的占位动画	13
9.2 回到顶部按钮	13
9.3 页面切换动画	13
10 总结	13

1 概述

我在项目中负责前端基础设施和用户交互模块的开发。具体来说：

- 前端项目从零搭建 (Vue3 + TypeScript + Vite)
- 登录、注册、找回密码页面
- 消息聊天系统 (站内信 + 通知)
- 订单详情页面 (支付、发货、收货、评价全流程)
- Axios 网络请求封装 (Token 自动管理)
- 后端 5 个 Bug 修复

以下按模块逐一说明做了什么、怎么做的、为什么这么做。

2 前端项目如何启动和运行

2.1 项目是怎么组织起来的

当你打开浏览器访问 `http://localhost:3000`，发生了什么：

1. Nginx 收到请求，返回 `index.html` (一个几乎空的 HTML 文件)
2. HTML 中引用了一个 `<script>` 标签，指向打包后的 JavaScript 文件
3. 浏览器下载并执行这个 JS 文件，Vue 框架启动
4. Vue Router 读取当前 URL，决定渲染哪个页面组件
5. 页面组件发出 API 请求获取数据，渲染到屏幕上

这叫做 单页应用 (SPA) —— 页面不会整体刷新，只替换变化的部分。

2.2 项目的文件结构

每个文件夹有明确的职责：

`src/`

- `api/index.ts` → 所有后端接口的封装 (30+ 个 API 函数)
- `router/index.ts` → URL 和页面的映射关系
- `stores/user.ts` → 登录状态管理 (全局共享)
- `utils/request.ts` → Axios 实例 + 拦截器 (请求前自动加 Token)

utils/auth.ts → localStorage 读写工具（存 Token）
views/ → 页面组件（每个 .vue 文件是一个页面）
layouts/ → 布局组件（导航栏+页脚公共结构）

3 Axios 封装：所有网络请求的基础

这是最重要也最容易被忽略的基础设施。项目中所有后端通信都经过这一个入口。

3.1 什么是 Axios

Axios 是一个 HTTP 客户端库。简单来说，它负责在浏览器和服务器之间发送和接收数据。比如你点击“登录”按钮，前端通过 Axios 把用户名密码发给后端，后端返回一个 Token，前端再把这个 Token 存起来。

3.2 为什么要封装

直接使用 Axios 的话，每次发请求都要手动写：

- 请求的基础 URL（/api/v1）
- 请求头中的 Token（Authorization: Bearer xxx）
- 错误处理（401 过期、403 无权、500 服务器错）

封装之后，这些逻辑只写一次，所有页面共享。

3.3 核心代码和解释

```
// 创建一个 Axios 实例，配置基础行为
const request = axios.create({
  baseURL: '/api/v1',           // 所有请求都从这个路径开始
  timeout: 15000,               // 超过 15 秒自动取消请求
  headers: { 'Content-Type': 'application/json' }
})

// 请求拦截器：每次发请求之前自动执行
request.interceptors.request.use(config => {
  const token = getToken()      // 从 localStorage 读取 Token
  if (token) {
    // 把 Token 放进请求头，格式是 "Bearer <token>"
    config.headers.Authorization = `Bearer ${token}`
  }
})
```

```
}  
  return config  
})
```

通俗理解：拦截器就像一个门卫，每次有人要出门（发请求），门卫自动检查有没有带钥匙（Token），有就帮你挂在身上，没有就空手出门。

3.4 Token 自动刷新：最精巧的部分

Token 有有效期（2 小时）。过期后需要拿 Refresh Token 去换新的。关键问题是：如果同时有 3 个请求都遇到 Token 过期，要不要刷新 3 次？当然不要。只需要刷新 1 次，刷新成功后 3 个请求都用新 Token 重试。

```
let isRefreshing = false // 标志位：是否正在刷新  
let failedQueue = []      // 等待队列  
  
// 当收到 401（未授权）时  
if (error.response?.status === 401) {  
  if (isRefreshing) {  
    // 如果已经在刷新了，把请求放进队列等待  
    return new Promise((resolve) => {  
      failedQueue.push({ resolve })  
    }).then(token => {  
      // 刷新成功后，用新 Token 重试  
      originalRequest.headers.Authorization = `Bearer ${token}`  
      return request(originalRequest)  
    })  
  }  
  isRefreshing = true  
  // 发起刷新请求...  
}
```

这个模式叫“请求队列排队”。类比：三个人同时发现门锁坏了（Token 过期），第一个去拿新钥匙，其余两个在旁边排队等着，新钥匙拿回来后三个人都用新钥匙进门。

4 登录与认证流程

4.1 完整登录流程

1. 用户在登录页输入用户名/邮箱/手机号 + 密码

2. 前端发送 POST /api/v1/user/login, 请求体包含 {account, password}
3. 后端验证密码 (用 BCrypt 比对哈希值), 成功则返回两个 Token:
 - Access Token: 2 小时有效期, 用于日常请求的身份验证
 - Refresh Token: 7 天有效期, 仅用于刷新 Access Token
4. 前端把两个 Token 存到 localStorage (浏览器的本地存储, 关闭页面也不会丢失)
5. 前端把用户信息 (昵称、头像、角色) 也存到 localStorage
6. 后续所有请求, 前端自动在请求头中携带 Access Token
7. 后端 JwtAuthenticationFilter 拦截每个请求, 解析 Token 获取用户 ID, 从数据库查用户状态, 设置认证上下文

4.2 为什么需要两个 Token

单 Token 的问题: 如果有效期短 (比如 30 分钟), 用户频繁操作时会不断掉线; 如果有效期长 (比如 30 天), Token 一旦泄露危害时间长。

双 Token 方案: Access Token 短 (2h) 但高频使用, Refresh Token 长 (7d) 但只在刷新时用。这样泄露 Access Token 的危害窗口只有 2 小时, 而用户体验不受影响 (自动刷新)。

4.3 登出时的安全处理

用户点击退出登录时, 不只是清除前端的 Token:

1. 前端发送 POST /api/v1/user/logout
2. 后端把当前 Token 的 ID 加入 Redis 黑名单, TTL 设置为 Token 的剩余有效时间
3. 后续请求中, JwtAuthenticationFilter 会检查 Token 是否在黑名单中
4. 即使在有效期内的 Token, 如果被加入了黑名单, 也会被拒绝

这解决了 Token 无法主动失效的问题 (JWT 是无状态的, 服务器无法在签发后撤销它, 除非用黑名单)。

5 注册与验证码机制

5.1 注册流程

1. 用户输入用户名、邮箱、密码
2. 点击”获取验证码”,前端发送 POST /api/v1/user/code,请求体 {target: email, type: "register"}
3. 后端生成 6 位随机数字,存到 Redis, Key 为 captcha:register: 邮箱, TTL 5 分钟
4. 后端通过 JavaMailSender 发送邮件到用户邮箱
5. 用户输入收到的验证码,点击注册
6. 前端发送 POST /api/v1/user/register,携带用户名、密码、邮箱、验证码
7. 后端从 Redis 读取验证码比对,通过后用 BCrypt 加密密码存储到 user 表
8. 注册成功后删除 Redis 中的验证码

5.2 验证码的安全设计

三个层面的安全措施:

1. **类型隔离**: Redis Key中包含类型前缀 (register/reset), 注册验证码不能用于重置密码, 反之亦然。这是因为我发现原始代码的验证码 Key 没有类型区分。
2. **频率限制**: 同一邮箱每 5 分钟最多发 3 次验证码, 超过返回 429 (请求过多)。防止恶意刷验证码。
3. **防消费**: 验证码只在业务流程真正成功后删除。如果用户输入了正确的验证码但密码不符合要求, 验证码不会被消耗, 用户可以修改密码后重试。

5.3 验证码防消费的实现细节

这是我从 Bug 修复中总结的经验。原始代码的顺序是:

1. 从 Redis 读取验证码 → 比对 ← 通过
2. 删除 Redis 中的验证码 ← 删了!
3. 从数据库查用户 ← 通过
4. 检查新旧密码是否相同 ← 失败! 抛异常

第 4 步失败后，验证码已经在第 2 步被删除了。用户重试时需要重新获取验证码，体验极差。

修复后的顺序：把第 2 步（删除验证码）移到整个方法的最后。只有当密码真正更新成功后才删除。这样中间任何一步失败，用户都可以直接重试而不需要重新获取验证码。

6 消息系统

6.1 数据流

消息系统的数据流是这样的：

1. 用户 A 在聊天输入框输入文字，点击发送
2. 前端调用 `POST /api/v1/message/send`，请求体 `{toUserId, content, type:"text"}`
3. 后端 `MessageServiceImpl.sendMessage` 插入一条记录到 `message` 表 (`fromUserId, toUserId, content, isRead=0`)
4. 同时创建一条 `Notification` 记录到 `notification` 表，内容是“XXX 给你发来一条消息”
5. 用户 B 的前端每 5 秒轮询一次 `GET /api/v1/message/conversation/{userId}`，发现新消息后更新界面

6.2 为什么用轮询而不是 WebSocket

WebSocket 是真正的“实时”通信（服务器主动推送），但需要额外的后端配置。对于这个项目，每 5 秒轮询一次的体验已经足够好——用户发消息后最多等 5 秒就能看到回复，对非即时通讯场景完全可接受。而且轮询方案更简单、更稳定、更容易调试。

6.3 轮询的具体实现

```
// 在组件挂载时启动 5 秒定时器
onMounted(() => {
  poller = setInterval(pollMessages, 5000)
})

// 在组件卸载时清除定时器（否则离开页面后还在请求！）
onUnmounted(() => {
  if (poller) clearInterval(poller)
```

```
})

async function pollMessages() {
  // 只有打开了某个会话时才轮询（避免无效请求）
  if (!selectedUserId.value) return

  const res = await messageApi.getConversation(selectedUserId.value
    , {page:1,size:50})
  const newMsgs = res.data || []

  // 如果消息数量变了，说明有新消息，更新界面
  if (newMsgs.length !== messages.value.length) {
    messages.value = [...newMsgs].reverse()
    loadConversations() // 同时刷新会话列表
  }
}
```

6.4 商品页联系卖家的实现

商品详情页有一个“联系卖家”按钮。点击后的流程：

1. 检查是否登录，未登录跳转登录页
2. 跳转到 `/message?to= 卖家 userId`
3. 消息页面读取 URL 参数 `route.query.to`，自动打开与该卖家的聊天窗口
4. 即使之前没聊过天（没有会话记录），也能直接开始新对话

这个设计的巧妙之处在于：URL参数携带了“我要和谁聊天”的信息，消息页面无需额外的 API 调用就能确定聊天对象。

6.5 通知系统的实现

头部导航栏的铃铛图标是一个 `el-popover` 组件（点击弹出浮动面板）。每 30 秒轮询 `GET /api/v1/notification/unread/count` 更新红点数字。点击铃铛时调用 `GET /api/v1/notification/list` 获取最近 10 条通知。

```
// 通知轮询（每30秒）
async function refreshUnread() {
  const nRes = await notificationApi.getUnreadCount()
  notifUnread.value = nRes.data?.count || 0
}
```

```
}  
// 点击通知后跳转到对应页面  
async function readNotif(n) {  
  await notificationApi.markRead(n.id)  
  if (n.type === 'order') router.push('/order/list')  
  else router.push('/message')  
}
```

通知的创建是由后端自动触发的。我在 `MessageServiceImpl` 和 `OrderServiceImpl` 中添加了通知创建逻辑：发消息 → 创建通知、下单 → 通知卖家、付款 → 通知卖家、发货 → 通知买家。

7 订单详情页面

7.1 角色判断：你怎么知道我是买家还是卖家

订单同时涉及两个人：买家（下单的人）和卖家（发布商品的人）。页面需要根据当前登录用户来判断应该显示什么按钮。

```
const currentUser = getUserInfo() // 从 localStorage 读取当前登录  
    用户  
  
// 判断是不是买家：当前用户的 ID 等于订单的 buyerId  
const isBuyer = computed(() => currentUser?.userId === order.value  
    ?.buyerId)  
  
// 判断是不是卖家：当前用户的 ID 等于订单的 sellerId  
const isSeller = computed(() => currentUser?.userId === order.value  
    ?.sellerId)
```

这两个计算属性会随着 `order.value` 的变化自动更新。Vue 的 `computed` 是响应式的——当它依赖的数据变化时，它自动重新计算。

7.2 根据状态和角色决定显示什么

```
<!-- 买家 + 待付款 = 显示 "去支付" 和 "取消订单" -->  
<div v-if="isBuyer && order.status === '待付款'">  
  <el-button @click="goToPay">去支付</el-button>  
  <el-button @click="cancel">取消订单</el-button>  
</div>
```

```
<!-- 卖家 + 待发货 = 显示 "录入物流发货" -->
<div v-if="isSeller_&&_order.status_===_ '待发货' ">
  <el-button @click="showShipDialog=_true">录入物流发货</el-button>
</div>

<!-- 买家 + 待收货 = 显示 "确认收货" -->
<div v-if="isBuyer_&&_order.status_===_ '待收货' ">
  <el-button @click="receive">确认收货</el-button>
</div>

<!-- 买家 + 已完成 + 还没评价 = 显示 "去评价" -->
<div v-if="isBuyer_&&_order.status_===_ '已完成' _&&_!reviewed">
  <el-button @click="showReviewDialog=_true">去评价</el-button>
</div>
```

Vue 的 `v-if` 指令根据条件决定是否渲染这段 HTML。如果条件不满足，这段 HTML 根本不会出现在页面上。

7.3 支付页面：模拟支付的四步流程

支付页面不是真正的支付（没有对接微信/支付宝），而是对支付流程的前端模拟。它的价值在于：让项目演示有完整的交易闭环。

1. **选择支付方式**：展示三种方式（微信、支付宝、校园卡），用不同颜色的卡片区分。用户选择后点击“确认支付”。
2. **处理中**：显示旋转动画，用 `setTimeout` 模拟 1.5-2.5 秒的支付处理时间。然后调用 `PUT /api/v1/order/{id}/pay`。
3. **成功**：SVG勾选动画 + 价格确认 + 2 秒倒计时自动跳转。
4. **失败**：抖动动画 + 错误信息 + 重试按钮。

状态管理使用一个 `step` 响应式变量（`'select' → 'processing' → 'success'/'error'`），不同步骤渲染不同 UI。

7.4 发货功能

卖家点击“录入物流发货”后，弹出对话框，选择物流公司（8 个选项）并输入快递单号。前端调用 `PUT /api/v1/order/{id}/ship`，请求体 `{logisticsCompany, logisticsNo}`。后端更新订单状态为“待收货”，记录发货时间，创建订单日志，通知买家。

8 后端 Bug 修复

8.1 验证码类型隔离（安全漏洞）

问题：原始代码中，验证码存储在 Redis 中的 Key 是 `captcha: 邮箱`，没有区分是注册验证码还是重置密码验证码。攻击者可以先注册一个账号（获取注册验证码），然后用这个验证码去重置别人的密码。

修复：将 Redis Key 改为 `captcha: 类型: 邮箱格式`。注册时 Key 为 `captcha:register:xxx@qq.com`。重置时 Key 为 `captcha:reset:xxx@qq.com`。两者互不可用。

8.2 密码重置暴力破解防护

问题：6 位数字验证码只有 100 万种可能，如果无限次尝试，理论上可以暴力破解。原始代码的 `resetPassword` 方法没有任何频率限制。

修复：同一邮箱每 15 分钟最多尝试 5 次。使用 Redis Key `pwd:reset:rate: 邮箱` 计数，超过限制返回 429。

8.3 浏览量统计错误

问题：卖家查看自己发布的商品也增加浏览量，导致数据不真实。

修复：在 `ProductServiceImpl.getProductDetail` 中增加判断——如果请求用户是该商品的发布者，则跳过浏览量 +1。

8.4 订单 VO 缺少字段

问题：后端返回给前端的订单数据中没有 `buyerId` 和 `sellerId`，前端无法判断当前用户是买家还是卖家。

修复：在 `OrderVO` 中增加 `buyerId` 和 `sellerId` 字段，在 `OrderServiceImpl` 中赋值。

8.5 订单状态变更自动通知

问题：订单状态变更后（付款、发货、收货），相关方（买家/卖家）没有任何通知，只能手动刷新页面查看。

修复：在 `OrderServiceImpl` 的各状态流转方法中增加通知创建逻辑。封装了 `notifyUser(userId, title, content)` 方法统一处理。

9 页面动效的实现

9.1 骨架屏：加载中的占位动画

当页面正在请求数据时，如果显示一片空白，用户会觉得卡住了。骨架屏预先画好几个灰色的卡片形状，提示用户“内容正在加载中”。

实现原理：用 CSS animation 做一个从左到右移动的光效（渐变色背景 + 移动 background-position）。数据加载完成后，v-if 切换到真实内容。

9.2 回到顶部按钮

监听 window.scrollY, 超过 400 像素时显示一个圆形按钮, 点击后调用 window.scrollTo({top: 0, behavior: 'smooth'}) —— 浏览器原生平滑滚动。按钮的显示/隐藏用 CSS transition 做弹性缩放。

9.3 页面切换动画

Vue Router 的 <router-view> 支持 <transition> 包裹。设置 mode="out-in" 表示旧页面完全离开后新页面再进入，避免两个页面同时出现在屏幕上。动画效果用 CSS transition 实现——旧页面淡出，新页面从下方 16px 处淡入上滑。

10 总结

这个项目中我写的代码主要解决了以下问题：

- **前端如何与后端通信**：通过 Axios 封装，自动处理 Token 注入、过期刷新、错误分类
- **用户如何登录和保持登录状态**：JWT 双 Token 方案，Access Token 2h + Refresh Token 7d，自动刷新
- **验证码如何保证安全**：类型隔离（不同用途不通用）+ 频率限制（防暴力）+ 防消费（失败不删除）
- **消息如何实时更新**：前端轮询（每 5 秒拉一次），比 WebSocket 简单但效果足够
- **订单流转如何控制**：根据“当前用户角色 × 订单状态”两个维度决定显示什么操作按钮
- **支付流程如何模拟**：四步状态机（选择 → 处理中 → 成功/失败），纯前端实现

11 深入：JWT 的工作原理

JWT (JSON Web Token) 是 JSON 格式的令牌，用于在不同系统之间安全地传递信息。本项目的认证完全基于 JWT 实现。

11.1 一个 JWT 长什么样

一个完整的 JWT 由三个部分组成，用点号连接：`header.payload.signature`。例如：

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjMONTY3ODkwIn0.dozjgNryP4J3jVmNH10w5N_XgL0n3I9PlFU
```

↑ header ↑ payload ↑ signature

解码后：

- **Header (头部)**: `{"alg": "HS256", "typ": "JWT"}`。说明签名算法是 HMAC-SHA256。
- **Payload (载荷)**: `{"sub": "1234567890", "iat": 1516239022, "exp": 1516242622}`。包含用户 ID (sub)、签发时间 (iat)、过期时间 (exp) 等声明。
- **Signature (签名)**: 用 Header 指定的算法，把 Header + Payload + 密钥一起计算出的哈希值。用于验证 Token 没有被篡改。

11.2 为什么 JWT 是安全的

关键在于签名。如果有人修改了 Payload 中的用户 ID (比如从 123 改成 456)，签名就对不上了。因为签名是用原始 Payload + 密钥计算的，修改后的 Payload 算不出相同的签名 (除非知道密钥)。

但 JWT 的 Payload 只是 Base64 编码 (不是加密)，任何人解码后都能看到内容。所以 JWT 不能存放敏感信息 (如密码)。Base64 编码和解码是公开算法，不依赖密钥。

11.3 为什么需要签名验证

考虑这个场景：服务端签发了 Token A (用户 ID=123，签名=abc)。攻击者截获了 Token A，把 Payload 中的用户 ID 改成 456，签名保持不变。服务端收到被篡改的 Token 后，用密钥对“Header+ 被篡改的 Payload”重新计算签名，得到 xyz。因为 xyz ≠ abc，服务端知道 Token 被篡改，拒绝访问。

这就是为什么 JWT 能防止篡改：签名的不可伪造性来自于密钥的保密性。只要密钥不泄露，任何人都无法伪造有效的 JWT。

11.4 Access Token 和 Refresh Token 的区别

本项目使用双 Token 策略：

Access Token：

- 有效期短（2 小时）
- 用于日常 API 请求的身份验证
- 每次请求都在 Authorization 头中携带
- 泄露后的危害窗口小（最多 2 小时）

Refresh Token：

- 有效期长（7 天）
- 仅用于获取新的 Access Token
- 只在 Access Token 过期时使用
- 泄露后危害大（7 天内的所有请求都可伪造）

这种设计的核心思想是”最小暴露”：高频使用的 Token 短效，低频使用的 Token 长效。攻击者即使截获了 Access Token，也只能在 2 小时内使用。

11.5 Token 黑名单：解决”无法撤销”的问题

JWT 的一个固有问题：服务端签发了 Token 后，无法主动让它失效。因为 JWT 是无状态的——服务端不存储 Token 信息，只靠签名验证真伪。

解决方法是维护一个黑名单（本项目使用 Redis）。用户主动退出登录时，服务端把当前 Token 的 ID（jti 字段）加入 Redis，设置过期时间等于 Token 的剩余有效期。后续请求的 JWT 过滤器会先检查黑名单，如果 Token 在黑名单中，即使签名有效也拒绝访问。

为什么 Redis 适合做黑名单：Redis 支持 TTL（过期时间），Token 有效期到了自动从黑名单删除，不需要手动清理过期数据。

12 深入：BCrypt 密码哈希

12.1 为什么不能明文存密码

如果数据库以明文存储密码，一旦数据库泄露，所有用户的密码都暴露了。而且大部分用户在不同网站使用相同密码，一个网站泄露会导致多个网站的账号被盗。

12.2 为什么不能只做一次哈希

$\text{SHA256}("123456") = "8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92"$ 。这确实是不可逆的（从哈希值无法反推原密码），但为什么还不够安全？

因为攻击者可以预计算常见密码的哈希值（彩虹表攻击）。比如提前算好“123456”、“password”、“admin123”等百万个常见密码的 SHA256 值，然后暴力匹配。

12.3 BCrypt 如何解决

BCrypt 做了两件事：

加盐 (Salt)：每次哈希时生成一个随机字符串（盐），拼接到密码后面再哈希。这样即使两个用户使用相同的密码，哈希结果也不同。攻击者无法使用彩虹表（因为每个密码的盐都不同）。

慢哈希：BCrypt 故意设计成计算很慢（通过 cost factor 控制迭代次数）。普通 SHA256 一秒可以计算数百万次，BCrypt 一秒只能计算几次。这大大增加了暴力破解的时间成本——原本 1 秒能试完的密码组合，现在需要数年。

本项目使用 BCryptPasswordEncoder，默认 cost factor 为 10，每次哈希需要约 0.1 秒。

12.4 Spring Security 如何集成 BCrypt

当用户注册时，`passwordEncoder.encode("明文密码")` 返回一个 60 字符的哈希字符串。当用户登录时，`passwordEncoder.matches("用户输入的密码", "数据库中存的哈希")` 自动从哈希中提取盐值，用相同的参数重新计算哈希并比对。比对过程不需要知道原始密码。

13 深入：Vue 的响应式系统

13.1 什么是“响应式”

响应式是指：当数据变化时，依赖这个数据的界面自动更新。这是 Vue 框架最核心的特性，也是它能简化前端开发的根本原因。

举个例子：`const count = ref(0)` 创建了一个响应式变量。当你在代码中 `count.value = 5` 时，页面上所有显示 `count` 的地方都会自动变成 5。你不需要手动操作 DOM。

13.2 Vue 3 用 Proxy 实现响应式

Vue 2 使用 `Object.defineProperty` 实现响应式，但有一些限制：无法检测对象属性的添加和删除、无法监听数组索引和长度的变化。Vue 3 改用 JavaScript 的 Proxy API：

```
// 简化原理：
const data = { count: 0 }
const proxy = new Proxy(data, {
  get(target, key) {
    // 记录"谁在读取这个属性"
    track(target, key)
    return target[key]
  },
  set(target, key, value) {
    target[key] = value
    // 通知"所有读取过这个属性的地方，数据变了"
    trigger(target, key)
    return true
  }
})
```

当你访问 `proxy.count` 时，`get` 拦截器记录“这个地方依赖 `count`”。当你修改 `proxy.count = 5` 时，`set` 拦截器通知所有依赖 `count` 的地方重新计算。这就是响应式的本质——数据劫持 + 依赖收集 + 变更通知。

13.3 computed vs watch

computed：定义一个“计算属性”，它的值取决于其他响应式数据，当依赖变化时自动重新计算。适合“从已有数据推导出新数据”的场景。

```
// 全选状态：当所有项的 selected 都是 true 时，全选才为 true
const allSelected = computed(() =>
  list.value.length > 0 && list.value.every(i => i.selected)
)
```

watch：监听一个响应式数据的变化，当变化时执行副作用（如发网络请求）。适合“数据变化后做点什么”的场景。

14 深入：浏览器存储机制

14.1 localStorage vs sessionStorage vs Cookie

localStorage：数据存在浏览器中，关闭页面、关闭浏览器、重启电脑都不会丢失。容量约 5-10MB。只能存储字符串。本项目用它存储 Token 和用户信息。

sessionStorage：数据在关闭标签页后清除。其他和 `localStorage` 一样。

Cookie：每次 HTTP 请求自动附带（这是最大的区别）。容量只有 4KB。可以设置过期时间和 HttpOnly（JS 不可读）。传统 Web 应用的 Session ID 通常存在 Cookie 中。

14.2 为什么 Token 存在 localStorage 而不是 Cookie

Cookie 的 HttpOnly 属性虽然安全（JS 无法读取，防止 XSS 窃取），但本项目的前端需要在每次请求时手动读取 Token 并加到请求头中。如果存在 HttpOnly Cookie 中，JS 读不到。

Cookie 会自动附加到每次请求，这也有问题：跨域请求可能被浏览器拦截。而用 Authorization 头手动添加 Token，不受同源策略影响。

14.3 XSS 攻击如何窃取 localStorage

如果攻击者在页面上注入了恶意的 `<script>` 标签，其中包含 `localStorage.getItem('accessToken')` 就能偷走 Token。所以防御 XSS 和正确存储 Token 是相辅相成的安全措施。

本项目的前端防御：商品描述使用 `sanitizedDescription` 过滤 script 标签。后端的 Content-Security-Policy 头（通过 Nginx 配置）限制可执行的脚本来源。

15 深入：HTTP 协议基础

15.1 GET 和 POST 的区别

GET：从服务器获取数据。参数放在 URL 后面（query string）。浏览器会缓存 GET 请求的结果。GET 请求应该是幂等的（多次执行结果相同），不应有副作用。例如：获取商品列表、获取用户信息。

POST：向服务器提交数据。参数放在请求体中（body）。不会被缓存。通常用于创建、修改、删除等有副作用的操作。例如：注册、登录、发布商品、下单。

PUT vs POST：PUT 用于完整替换已有资源（幂等），POST 用于创建新资源（非幂等）。本项目遵循 REST 规范：更新操作用 PUT，创建操作用 POST。

15.2 HTTP 状态码的含义

- 200 OK：请求成功
- 201 Created：创建成功（如注册成功后返回 201）
- 400 Bad Request：请求参数有误（客户端的问题）
- 401 Unauthorized：未认证（没登录或 Token 过期）

- 403 Forbidden: 已认证但无权限 (不是管理员)
- 404 Not Found: 资源不存在
- 409 Conflict: 冲突 (如用户名已存在)
- 429 Too Many Requests: 请求频率过高
- 500 Internal Server Error: 服务器内部错误

前端根据状态码做不同的错误处理, 这就是 Axios 响应拦截器中错误分类逻辑的依据。

16 深入：前端路由原理

16.1 Hash 模式 vs History 模式

单页应用 (SPA) 的核心问题: 如何在不刷新页面的情况下, 让 URL 变化并显示不同的内容?

Hash 模式: URL 中使用 # 符号。例如 `/index.html#/login`。# 后面的部分不会发送到服务器, 由前端 JS 读取并决定渲染哪个页面。兼容性好但 URL 不美观。

History 模式: 使用 HTML5 History API (`pushState/replaceState`), URL 看起来像正常的路径: `/login`。本项目使用 History 模式, 但需要 Nginx 配合——所有路径都返回 `index.html`, 否则用户直接访问 `/login` 时 Nginx 会返回 404。

16.2 Vue Router 的导航守卫

```
router.beforeEach((to, from, next) => {
  // 如果目标页面要求登录, 但用户没登录
  if (to.meta.requiresAuth && !isLoggedIn()) {
    next('/login')    // 重定向到登录页
    return
  }
  // 如果目标页面要求管理员权限
  if (to.meta.requiresAdmin) {
    const user = getUserInfo()
    if (!user || user.role !== 'admin') {
      next('/')        // 重定向到首页
      return
    }
  }
})
```

```
next() // 放行  
})
```

导航守卫在每次路由变化前执行，类似一个门禁系统。每个路由可以在 `meta` 中定义权限要求，守卫统一检查。

16.3 路由懒加载

本项目的路由配置使用了动态 `import`：

```
{ path: 'product/list', component: () => import('@views/product/  
  ProductList.vue') }
```

这行代码的意思是：只有当用户真正访问这个路径时，才去下载对应的 JS 文件。初始加载时只下载首页需要的代码。这减少了首次加载的包体积，加快页面打开速度。Vite 构建时会自动将动态 `import` 的模块分离成独立的 `chunk` 文件。